MODULE 15 · DOMAIN 2 - SECURITY

IAM Policies and How Access Is Decided

Six policy types, one evaluation algorithm, and the two exceptions that catch everybody.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 15 of 34

admin@sthithapragna.com

Every AWS API call is **denied by default**. Something must explicitly allow it, and nothing may explicitly deny it.

Users, groups and roles

- A **user** is a long-lived identity with permanent credentials. A person or, historically, an application.
- A **group** is a container for users. It holds policies. **A group is not an identity** and cannot be a principal.
- A **role** has no permanent credentials. It is assumed, yielding temporary ones.
- Roles are assumed by AWS services, by other accounts, by federated users and by EC2 instances.
- For applications, the answer is **always a role**. Never an access key stored in code or on a server.

> ON THE EXAM
>
> That last line is close to an absolute on this exam. Any option involving embedded access keys is essentially always wrong.

The root user

- The root user has unrestricted access and **cannot be limited by an identity policy**.
- Enable **MFA** on it, remove any access keys, and then stop using it.
- A handful of tasks genuinely require it: closing the account, changing support plans, some billing settings.
- An **SCP** is the only thing that can constrain root, and only in a member account.
- Everything else should use an IAM identity or, better, a federated role.

> ON THE EXAM
>
> Any option that solves a problem by using root credentials is wrong. The root user is for a short list of account-level tasks and nothing else.

The six policy types

TYPE	WHAT IT DOES
Identity-based	Attached to a user, group or role. Grants permissions
Resource-based	Attached to a resource. Grants permissions, and names a Principal
Permissions boundary	Caps what an identity policy can grant. Never grants
Service control policy	Caps what an account may do, across an Organization. Never grants
Session policy	Caps a specific assumed-role session. Never grants
Access control list	Legacy, cross-account, on S3 and a few others. Disabled by default now

ON THE EXAM

Only two of these grant anything. The middle three are ceilings, and confusing a ceiling with a grant is the most common IAM misunderstanding.

Anatomy of a policy

- **Version** is a date, and it should be 2012-10-17. Older versions lack policy variables.
- **Effect** is Allow or Deny. **Action** names the API operations. **Resource** names the ARNs.
- **Principal** appears only in resource-based policies. It names who the policy is about.
- **Condition** makes the statement apply only when the request context matches.
- **Sid** is an optional label. **NotAction** and **NotResource** invert the match, and are easy to get wrong.

> ON THE EXAM
>
> The presence or absence of Principal is the quickest way to tell the two granting policy types apart when you are shown a policy document.

Reading a policy quickly

- Find **Effect** first. A Deny statement matters more than every Allow around it.
- Then **Resource**. A wildcard here is usually where the over-permission lives.
- Then **Condition**. It is where a policy that looks permissive becomes narrow.
- Check for **Principal** to identify whether it is a resource-based policy.
- Watch for **NotAction**, which means everything **except** what is listed. It is easy to misread as a whitelist.

> ON THE EXAM
>
> Exam questions show policies you must read under time pressure. Effect, Resource, Condition, in that order, extracts the meaning fastest.

Actions, resources and wildcards

- Actions are **service:Operation**, such as s3:GetObject or dynamodb:PutItem.
- Wildcards work: **s3:Get*** matches every S3 read operation.
- **Action: "*"** with **Resource: "*"** is administrator access. It is almost never a correct answer.
- An ARN is **arn:partition:service:region:account-id:resource**.
- S3 and IAM ARNs omit region and account, which is why S3 bucket ARNs look shorter.

> ON THE EXAM
>
> For S3, the bucket and the objects are two different ARNs. Listing needs the bucket ARN; reading objects needs the bucket ARN with a slash-star suffix.

How a request is decided

01

Deny by default

Every request starts as an implicit deny.

02

Any explicit Deny?

Check every applicable policy. One Deny ends it.

03

Within the ceilings?

SCPs, permissions boundaries and session policies must all permit it.

04

Any explicit Allow?

An identity or resource policy must grant it. Otherwise the implicit deny stands.

ON THE EXAM

Four steps, in this order. Explicit deny is the only universal short-circuit in IAM: nothing anywhere can override it.

Implicit deny against explicit deny

- **Implicit deny** is the default. Nothing granted the permission.
- **Explicit deny** is an actual Deny statement that matched the request.
- Both surface as **AccessDenied**, so the error message does not tell you which.
- The fix differs entirely: implicit means add a grant, explicit means find and remove the block.
- Adding permissions can never resolve an explicit deny. That is the diagnostic tell.

> ON THE EXAM
>
> 'We gave them administrator access and it still fails' is the signature of an explicit deny, almost always from an SCP or a permissions boundary.

Same-account evaluation

- Within one account, the identity policy and the resource policy are a **union**.
- If **either** allows the action, and neither denies it, the request succeeds.
- So a bucket policy alone can grant access to a principal in the same account.
- And an identity policy alone can grant access to a bucket with no bucket policy.
- An explicit deny in either one still overrides the other's allow.

> ON THE EXAM
>
> This is why removing a bucket policy sometimes changes nothing: the identity policy was already granting the access independently.

Cross-account evaluation

- Across accounts, AWS runs **two separate evaluations**, one in each account.
- **Both must return Allow.** The union rule does not apply.
- The **trusted** account's identity policy must allow the action on that resource.
- The **trusting** account's resource policy must allow that principal.
- Granting only one side is the most common cross-account mistake there is.

> READ THIS TWICE
>
> Same account is OR. Cross account is AND. That single sentence resolves a large share of IAM questions on this exam.

Two ways to share across accounts

- **Assume a role** in the other account. The caller becomes a principal there.
- Permissions then come from the role, and CloudTrail in that account records the activity.
- Or use a **resource-based policy** naming the external principal directly.
- No role switch is needed, and the caller keeps its own identity throughout.
- Not every service supports resource-based policies. S3, SQS, SNS, KMS and Lambda do.

ON THE EXAM

Role assumption is the more general answer, because it works for every service. Resource policies are simpler when the service supports them.

Two exceptions to the OR rule

- **IAM role trust policies** must explicitly allow the principal. An identity policy alone is never enough.
- **KMS key policies** must explicitly allow access, even within the same account.
- So a KMS key with an empty key policy is unusable by anyone except the root account.
- Granting kms:Decrypt in an identity policy achieves nothing if the key policy does not permit it.
- Other services may have similar requirements. Check the service documentation.

> ON THE EXAM
>
> This is why KMS troubleshooting is its own skill. Module 18 returns to it, but the reason lives here in the evaluation logic.

Managed or inline policy?

MANAGED POLICY

A standalone object

- Attach to many identities at once
- Versioned, with rollback to a prior version
- AWS managed, or customer managed
- Reusable and centrally maintained
- The recommended default

INLINE POLICY

Embedded in one identity

- Belongs to exactly one identity
- Deleted when the identity is deleted
- No versioning at all
- Guarantees a strict one-to-one binding
- Useful when the policy must never be reused

THE TIEBREAKER

Prefer customer managed policies. AWS managed policies are convenient and usually far broader than least privilege requires.

Permissions boundaries

- A managed policy attached to a user or role that **caps** its maximum permissions.
- **It grants nothing.** Effective permissions are the intersection of the identity policy and the boundary.
- The classic use is **safe delegation**: let developers create roles, but not privileged ones.
- You require, via a condition, that any role they create carries the boundary.
- An action allowed by the identity policy but absent from the boundary is denied.

> ON THE EXAM
>
> This is how you delegate IAM without enabling privilege escalation. Without a boundary, anyone who can create roles can create an administrator.

Service control policies

- Applied through AWS Organizations to an account or an organizational unit.
- **Grants nothing.** It defines the maximum any principal in that account may do.
- **It applies to the root user too**, which is what makes it a genuine guardrail.
- Typical uses: deny Regions outside approved ones, deny disabling CloudTrail, deny deleting logs.
- It does not apply to the management account, which is why workloads should not live there.

> ON THE EXAM
>
> An SCP is the usual answer to 'the administrator cannot do it either'. Nothing inside the account can override it.

Permissions boundary or SCP?

PERMISSIONS BOUNDARY

Scope: one identity

- Attached to a user or role
- Set by whoever creates the identity
- Used to delegate IAM safely
- Does not affect other identities
- Works without Organizations

SERVICE CONTROL POLICY

Scope: a whole account

- Applied to an account or an OU
- Set centrally by the organization
- Used for organisation-wide guardrails
- Applies to every principal, root included
- Requires AWS Organizations

THE TIEBREAKER

One identity or one account. If the scenario says 'nobody in the account, including administrators', it is an SCP.

The Condition element

- A statement applies only when every condition in its Condition block evaluates true.
- Structure is **operator, then condition key, then value**.
- Multiple keys in one block are combined with **AND**. Multiple values for one key are **OR**.
- If the request context **lacks** the key, the condition fails and the statement does not apply.
- That last point makes **Deny with a condition** subtle, and it is a favourite exam trick.

> ON THE EXAM
>
> A Deny statement conditioned on a key that is absent simply does not apply, so the deny does not happen. This is how people accidentally write policies that do nothing.

Condition operators

OPERATOR	USE
`StringEquals, StringLike`	Exact match, or match with wildcards
`ArnEquals, ArnLike`	Compare ARNs, with wildcard support
`NumericLessThan, NumericGreaterThan`	Numeric comparison
`DateGreaterThan, DateLessThan`	Time-bounded access
`Bool`	True or false keys, such as aws:SecureTransport
`IpAddress, NotIpAddress`	CIDR range matching on the source IP
`Null`	Whether a key is present in the request at all
`...IfExists suffix`	Apply the check only when the key is present

ON THE EXAM

The IfExists suffix is the fix for the absent-key problem: the condition passes when the key is missing, rather than failing.

Global condition keys worth knowing

KEY	WHAT IT CONSTRAINS
aws:SecureTransport	Whether the request used HTTPS. Deny when false
aws:SourceIp	The caller's IP address. Not for AWS service calls
aws:SourceArn / aws:SourceAccount	Which resource or account triggered a service to call on your behalf
aws:PrincipalTag / aws:ResourceTag	Tags on the caller and on the resource. The basis of ABAC
aws:RequestTag	Tags being applied by this request
aws:MultiFactorAuthPresent	Whether MFA was used for these credentials
aws:PrincipalOrgID	Whether the caller belongs to your AWS Organization

ON THE EXAM

aws:PrincipalOrgID is the clean way to allow an entire organisation without listing account IDs. aws:SecureTransport false is the standard HTTPS-only deny.

Policy variables

- Substitute request context values into a policy at evaluation time.
- **${aws:username}** lets one policy give every user their own S3 prefix.
- **${aws:PrincipalTag/team}** substitutes a tag value from the caller.
- They require **Version 2012-10-17**. Older policy versions silently treat them as literal text.
- They work in Resource and in Condition values, not in Action.

> ON THE EXAM
>
> This is how you avoid writing one policy per user. A home-directory pattern in S3 is the standard example, and it appears in scenarios.

RBAC or ABAC?

ROLE-BASED, RBAC

A policy per job function

- Separate policies for separate roles
- Explicit and easy to audit
- Policy count grows with teams
- Every new team needs a change
- The traditional approach

ATTRIBUTE-BASED, ABAC

One policy, driven by tags

- Compares principal tags to resource tags
- One policy can serve every team
- New teams need no policy change
- Depends on disciplined tagging
- Scales to many teams cleanly

THE TIEBREAKER

The signal is scale. 'Hundreds of teams' or 'without creating a new policy each time' means ABAC. It also means tagging must be enforced.

Trust policies

- Every role has a **trust policy**, which is a resource-based policy naming who may assume it.
- The principal can be a service, an account, a specific role, or a federated provider.
- **Two things are needed:** the trust policy must allow it, and the caller must be allowed sts:AssumeRole.
- **ExternalId** in a third-party trust policy prevents the confused deputy problem.
- The role's own permissions come from a separate identity policy attached to it.

> ON THE EXAM
>
> Trust policy and permissions policy are two different documents on the same role. 'Who can wear this hat' against 'what the hat lets you do'.

Writing least privilege

- Start from an AWS managed policy to get working, then narrow it. Do not ship it.
- Use **IAM Access Analyzer policy generation** to build a policy from actual CloudTrail activity.
- Check **last accessed data** to find granted services that have never been used.
- Scope Resource to specific ARNs. A wildcard resource is where over-permission usually hides.
- Add conditions rather than removing actions, where the constraint is contextual.

> ON THE EXAM
>
> Policy generation from CloudTrail is the answer when a scenario asks how to build a least-privilege policy for an existing application.

IAM Access Analyzer

- **External access findings** identify resources shared outside your account or organisation.
- **Unused access findings** identify roles, users and permissions nobody has exercised.
- **Policy validation** checks a policy against grammar and best practice before you save it.
- **Custom policy checks** can fail a CI/CD pipeline when a change grants new access.
- Findings are produced by automated reasoning, not by sampling traffic.

ON THE EXAM

The custom policy check in a pipeline is worth knowing: it turns 'did this pull request widen permissions' into an automated test.

Diagnosing AccessDenied

- Read the message. It usually names the **principal**, the **action** and the **resource**.
- Is the resource in **another account**? Then both sides must allow it.
- Is there an **SCP or permissions boundary**? Broad permissions failing points there.
- Is a **condition** failing? Check for a missing key, or an IP or MFA requirement.
- Is it **KMS**? The key policy must allow it independently of the identity policy.

> ON THE EXAM
>
> Use the IAM policy simulator to test before deploying, and CloudTrail to see the exact denied call afterwards. The simulator does not cover SCPs.

Credential report and MFA

- The **credential report** is a CSV of every user, listing key age, password age and MFA status.
- It is the standard answer for auditing which users lack MFA or hold stale access keys.
- **aws:MultiFactorAuthPresent** lets a policy require MFA for sensitive actions.
- Use it with **BoolIfExists**, or calls made by AWS services will fail the condition.
- Access keys should be rotated regularly, and the report is how you find the ones that were not.

> ON THE EXAM
>
> The BoolIfExists detail matters: a plain Bool check on MFA denies service-made calls, because those requests carry no MFA key at all.

For each one, decide first: is this an **implicit** deny or an **explicit** deny?

> A developer's IAM role has the AdministratorAccess managed policy attached. They can work normally in eu-west-1, but every attempt to create resources in us-east-1 fails with AccessDenied.
>
> The account is a member of an AWS Organization. Adding further permissions to the role changes nothing.

What is the cause?

A. A service control policy on the account or its OU denies actions outside approved Regions

B. The AdministratorAccess policy does not cover us-east-1 and needs a Region condition added

C. The role is missing a trust policy for the us-east-1 endpoint

D. us-east-1 must be enabled for the account before it can be used

◆ PAUSE AND DECIDE

What is the cause?

A An SCP denying non-approved Regions

Administrator access failing, and more permissions not helping, is the signature of an explicit deny. An SCP is the only thing that can override AdministratorAccess.

B AdministratorAccess is Region-limited

It is Action star on Resource star with no Region condition. It covers every Region by definition.

C Missing trust policy

Trust policies govern who may assume a role, not which Regions it may act in. The role is clearly already assumed.

D Region not enabled

us-east-1 is enabled by default in every account. Some newer Regions are opt-in, but not this one.

THE SIGNAL

Two clues together: administrator permissions are failing, and adding more changes nothing. Both point at explicit deny, and in an Organization that means an SCP.

> Account A runs a Lambda function that must read objects from a bucket in Account B. The bucket policy in Account B has been updated to allow s3:GetObject with the Lambda execution role's ARN as the Principal. Block Public Access is enabled, and there are no Deny statements anywhere.
>
> The function still receives AccessDenied. The same function reads buckets in Account A without any problem.

What is missing?

A. Block Public Access must be disabled on the bucket in Account B

B. The bucket policy must use the Account B root as the Principal instead

C. The Lambda function must be moved into Account B

D. The Lambda execution role in Account A needs an identity policy allowing s3:GetObject on that bucket

◆ PAUSE AND DECIDE

What is missing?

A Disable Block Public Access

Block Public Access blocks anonymous and public access. Granting a named role in a named account is neither, so it is unaffected.

B Use the Account B root as Principal

That would grant access to Account B's own principals. The caller is in Account A, so this grants the wrong account.

C Move the function

It would work by turning this into a same-account request, and it is a large architectural change to avoid adding one policy statement.

D Add the identity policy in Account A

Cross-account access requires both sides. The resource policy allows it; the identity policy in the trusted account does not yet.

THE SIGNAL

The last sentence is the clue. Same-account reads work because identity OR resource is enough. Cross-account needs AND, and only one side is configured.

> A platform team wants application developers to create their own IAM roles for Lambda functions, so the platform team is not a bottleneck. Security requires that no role a developer creates can ever grant more than read access to S3 and write access to one DynamoDB table.
>
> Developers must not be able to grant themselves or their roles administrator access, directly or indirectly.

How should this be implemented?

A Give developers iam:CreateRole and review every role they create manually

B Apply a service control policy to the account limiting all principals to those permissions

C Grant iam:CreateRole with a condition requiring a permissions boundary, and attach that boundary to every role they create

D Create the roles in advance and let developers choose from a fixed list

◆ PAUSE AND DECIDE

How should this be implemented?

A CreateRole plus manual review

Manual review is not a control, it is a hope. A developer can create and use a privileged role before anyone looks at it.

B SCP on the account

An SCP would cap everyone in the account, including the platform team and any production workload. Far too blunt for delegation.

C CreateRole conditioned on a permissions boundary

Exactly what boundaries exist for. Developers create roles freely, and every role is capped, so privilege escalation is structurally impossible.

D Fixed list of pre-made roles

Removes the escalation risk by removing the delegation. The platform team is a bottleneck again, which is the problem being solved.

THE SIGNAL

The requirement is delegation plus a ceiling, on identities developers create themselves. That combination is the textbook permissions boundary use case.

> A company has around 200 project teams, each with its own EC2 instances, S3 buckets and DynamoDB tables. Each team must access only its own resources. New teams are created several times a week.
>
> The current approach is one IAM policy per team, and maintaining 200 near-identical policies has become unmanageable. All resources are already tagged with a Project key.

What should the company implement?

A. One IAM group per team, with the existing policies attached to groups instead of users

B. An ABAC policy comparing aws:PrincipalTag/Project to aws:ResourceTag/Project

C. A permissions boundary per team, applied to that team's roles

D. A separate AWS account per team, with cross-account roles

◆ PAUSE AND DECIDE

What should the company implement?

A Groups with the same policies

Improves how policies are attached, not how many exist. You still maintain 200 near-identical documents.

B ABAC comparing principal and resource tags

One policy serves every team, and a new team needs only correct tags rather than a new policy. Exactly the problem described.

C A boundary per team

Boundaries cap, they do not grant, so you would still need the 200 granting policies. It adds 200 more documents.

D An account per team

Genuinely used at scale, and an enormous change that adds 200 accounts to govern. Far beyond what the problem requires.

THE SIGNAL

'Already tagged with a Project key' is the planted detail. ABAC needs disciplined tagging, and the scenario tells you it exists.

What to carry forward

- **Explicit deny always wins.** If more permissions do not help, look for a deny.
- **Same account is OR. Cross account is AND.** Both sides must allow a cross-account call.
- **Trust policies and KMS key policies** must allow explicitly, even in the same account.
- Boundaries cap one identity. SCPs cap a whole account, including root. Neither grants.
- ABAC when the scenario says many teams, frequent change, or resources already tagged.

ON THE EXAM

Next: roles, temporary credentials and the credential chain - how an application actually obtains the permissions this module described.